MAMAN 14 · COURSE 20465 · ANSI C89/C90

How the Assembler Works

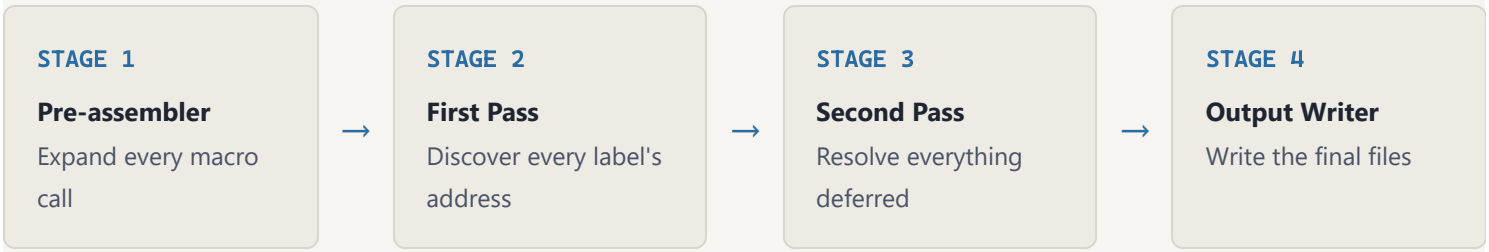
A complete structural and design-decision reference for the two-pass assembler project — the overall architecture, every module's purpose, and the specific reasoning (including the bugs we found and fixed) behind the way each piece is written.

01 The Core Problem, and Why the Design Looks Like This

An assembler's job sounds simple: read a line, translate it into machine code, write it out. The complication is **forward references**. A line like `jmp NEXT` can appear in the source file long before the line `NEXT: hlt` that defines where `NEXT` actually is. If you tried to translate the program in a single top-to-bottom read, you'd hit `jmp NEXT` with no way to know yet what address to encode.

The whole architecture exists to solve exactly this, and nothing more: **read the file twice.** The first read discovers where every label lives. The second read goes back with that complete picture and fills in everything the first read had to leave blank. Every module in this project ultimately supports one side or the other of that split.

On top of the two passes, there's a macro-expansion step before either of them runs (so passes 1 and 2 never have to think about macros at all — only plain assembly statements), and an output step after both succeed (turning the in-memory result into the three files the assignment actually asks for).




```
program.as → program.am → (symbol table + in-memory machine code) → program.ob / .ent / .ext
```

Underneath these four stages sits a layer of smaller, focused modules — a symbol table, a macro table, an opcode dictionary, string-parsing helpers, number-conversion helpers, and the raw memory image itself. None of them know anything about "passes" or "pipelines"; they're generic tools that the four stages above call into. That separation is deliberate: the tricky, error-prone bit-level and parsing logic lives in a small number of well-tested places, and the four pipeline stages are mostly just *orchestration* — deciding which tool to call, in what order, with which piece of the current line.

`assembler.c` is the thin layer on top of all of it: for every filename given on the command line, it runs the four stages in order and stops at the first one that fails, moving on to the next input file. Every stage resets its own internal state (symbol table, macro table, memory image, external-usage list) as the very first thing it does — so nothing from a previous file can leak into the next one in a multi-file run.

02 Foundational Modules

These eight modules don't belong to any one pipeline stage — they're the shared toolkit every stage draws on. Presented here as reference, not in any particular order (unlike the pipeline stages, there's no meaningful "sequence" to how these run).

`constants.h` shared limits & values	`errors` uniform error reporting
`opcode_table` the instruction dictionary	`symbol_table` the address book
`macro_table` macro definitions	`num_utils` text ↔ number ↔ bits
`line_parser` understanding one line	`memory_image` the simulated memory

`constants.h`

One source of truth for every fixed number the spec defines.

Line length limit, label length limit, the starting instruction address (100), file extensions, image array sizes — all defined once here rather than scattered as magic numbers through a dozen files. No `.c` file of its own; it's pure declarations, included wherever a module needs one of these values.

> **WHY A SEPARATE MAX_FILENAME_LENGTH FROM MAX_LINE_LENGTH**
>
> A file path and a source line are bounded by completely unrelated things in reality — reusing `MAX_LINE_LENGTH` (80, a rule about source lines) to size a filename buffer would have been a coincidence masquerading as a design decision, waiting to become a real bug the first time someone had a longer path.

`errors.h` / `errors.c`

The one place every other module goes to say "something's wrong."

Three functions: start tracking a new file (resets a counter), report one error with its line number, and ask whether the current file has had any errors yet. That last question is what lets the driver decide, after each stage, whether it's even worth continuing to the next one.

> **WHY FILE-SCOPE STATE INSTEAD OF PASSING A STRUCT EVERYWHERE**
>
> The alternative was threading an error-counter pointer through nearly every function in the whole project, for a program that only ever processes one file at a time. A small amount of static state, reset explicitly at the start of each file, was the simpler trade — simpler to read outweighed the theoretical purity of avoiding global-ish state here.

`opcode_table.h` / `opcode_table.c`

The dictionary of the language itself.

A static table of all 27 instructions — name, opcode, funct, whether it's R/I/J format, how many operands it takes — plus the 6 directive names. Two functions: look an instruction up by name, and ask "is this word off-limits as an identifier?" (covers instruction names, directive names, and eventually the macro keywords too).

> **WHY IS_RESERVED_WORD GREW A THIRD CATEGORY LATER**
>
> It originally only checked instruction and directive names. A later bug hunt found that a label literally named `mcro` could slip through undetected and then cause a confusing false alarm somewhere else entirely (in the pre-assembler's stray-word check, see below). The fix was adding `mcro` / `mcroend` to this same reserved-word check, so the problem is prevented at its actual source — a label can simply never be named that, full stop.

`symbol_table.h` / `symbol_table.c`

The address book: every label's name, address, and kind.

A dynamic linked list. Each entry: name, value (an address), kind (code / data / external), and an entry flag. Functions to add a symbol, look one up by name, mark one as an entry point, shift every data-symbol's value once the final code size is known, and walk the whole list (used later to build the `.ent` file).

> **WHY ADD_SYMBOL IS DELIBERATELY "DUMB"**
>
> It only knows one rule: fail if the name already exists. The real business logic — that redeclaring the same `.extern` symbol twice is fine, but declaring `.extern` on a name that's already a real label is an error — lives in `first_pass` instead, which checks *first* whether a symbol already exists and decides what that means before ever calling `add_symbol`. Keeping the table itself generic meant it didn't need to know anything about directive-specific rules.

> **WHY IT HAD TO SWITCH FROM PREPENDING TO APPENDING**
>
> New symbols were originally pushed onto the *front* of the list, which is simpler to write — but it means the list comes back out in reverse of definition order. That's invisible until something needs to walk the whole table in order, which is exactly what the `.ent` file writer does. Traced against the assignment's own worked example (which has two entry symbols, enough to actually expose the wrong order), the fix was a second, tail pointer — append at the tail instead of push at the head, the same technique already used in `macro_table` for a macro's body lines.

`macro_table.h` / `macro_table.c`

Holds every macro definition found while expanding one file.

Each macro: a name, and its body (a second, nested linked list of text lines). Add a macro, append a line to its body, look a macro up by name, free the whole thing between files.

> **WHY THE BODY LIST NEEDS A TAIL POINTER BUT THE TOP-LEVEL MACRO LIST DOESN'T**
>
> A macro's *lines* must come back out in the exact order they were written — line 1 of the macro, then line 2 — so appending needs a tail pointer for O(1) inserts. The list of *macros themselves* has no such requirement (nothing cares what order macros are stored in), so that one can just push onto the front, simpler still.

> **WHY A MACRO'S NAME FIELD IS SIZED DIFFERENTLY FROM A LABEL'S**
>
> It was originally sized the same as a label (31 characters) on the assumption macro names are "just identifiers like labels." But the assignment's own example macro name, `GEN_MAC`, contains an underscore — illegal in a label. Once it was clear macro names don't follow label syntax rules at all, capping their length at a label's limit was an unjustified, latent overflow risk for a legitimately longer macro name; the field was widened to a full line's length instead.

`num_utils.h` / `num_utils.c`

Everything about turning text into numbers, and numbers into bits.

Four functions, each a different stage of a number's life: `parse_integer` (is this text even a legal number?), `fits_in_bits` (is it small enough for where it's going?), `to_bits` (what's its actual bit pattern for a field of a given width?), `get_byte` (split a value into individual storable bytes).

THE 32-BIT OVERFLOW STORY — PART 1

The first version of `fits_in_bits` / `to_bits` computed field masks by shifting a plain signed `int`: for example `(1 << num_bits) - 1`. That's fine for small widths, but the `.dw` directive needs the *full* 32-bit range, meaning these functions get called with `num_bits = 32` — and shifting a signed int by 31 or 32 positions is **undefined behavior** in C, not just "a big number." It doesn't crash; it silently produces a wrong mask. The fix: special-case `num_bits >= 32` to skip the shift entirely, since a 32-bit field already covers every bit an `int` has — there's nothing left to mask.

THE 32-BIT OVERFLOW STORY — PART 2 (THE DEEPER BUG)

A later, more thorough audit found that even with that guard in place, `to_bits` still broke at exactly `num_bits == 31` — one value short of the guard's threshold — because its mask formula shifts by `num_bits` directly (not `num_bits - 1`), so 31 still overflows a signed int the same way 32 did. This one mattered concretely: instruction opcodes go as high as 63 (`hlt`), and building a 32-bit instruction word means shifting the opcode field into bits 31–26 — exactly where this bug lived. The fix was switching the mask's own arithmetic from signed to **unsigned** (`1u << num_bits`): shifting an unsigned value into its top bit is fully defined by the C standard (it just wraps predictably), where the identical shift on a signed type is explicitly undefined if the result can't be represented as that signed type.

WHY PARSE_INTEGER ACCUMULATES IN A LONG, WITH AN EARLY BAIL-OUT

The original version accumulated digits into a plain `int`, which has the same problem as above in a different shape: a long enough digit string silently overflows a signed `int`, which is undefined behavior. The fix widens the accumulator to `long` and adds a check — the moment the running value exceeds `INT_MAX`, the function rejects the token outright, before `long` itself could ever get close to overflowing. The one accepted trade-off: the single literal `-2147483648` (exactly `INT_MIN`) gets rejected as "too large," since the magnitude is accumulated before the sign is applied — a narrow, deliberately accepted gap rather than adding real complexity to handle one boundary value nobody will realistically type into a test file.

WHY GET_BYTE IS SAFE EVEN FOR NEGATIVE VALUES

Right-shifting a negative signed integer is *implementation-defined* in C (not guaranteed to sign-extend), which sounds like another landmine — but it isn't one here, because the function only ever keeps the low 8 bits afterward via `& 0xFF`. For the byte positions this function is ever actually called with (0 through 3), both possible shift behaviors land on the identical 8 bits in that position. The apparent risk turns out not to matter for how the function is actually used.

`line_parser.h` / `line_parser.c`

Understanding the shape of one line of source.

Classify a line as empty / comment / directive / instruction; pull an optional label off the front; validate a label's, register's, or quoted string's syntax; split a comma-separated operand list; plus two small shared word-splitting helpers reused by the pipeline stages.

WHY LABEL EXTRACTION AND LABEL VALIDATION ARE TWO SEPARATE FUNCTIONS

`extract_label` is deliberately "dumb": it copies out whatever text precedes a colon without judging whether it's a *legal* label at all. That lets the caller tell apart "there's no label on this line" from "there's a label-shaped thing here, but it's illegal" — two different situations that deserve different error messages. The separate validity check, `is_valid_label_name`, also gets reused later for label *operands* (a `jmp` target, an `.entry` name) that never go through label extraction at all.

WHY REGISTER PARSING REJECTS A LEADING SIGN EXPLICITLY

`is_valid_register` checks that the character right after `$` is a plain digit *before* ever calling `parse_integer` on the rest. Left to itself, `parse_integer` would happily accept a leading `+` or `-` — legal for a general integer, but the assembly language's register syntax has no such sign. Without this extra check, something like `$-5` could be silently misread rather than correctly rejected.

WHY GET_FIRST_WORD/SKIP_FIRST_WORD ENDED UP HERE, NOT IN THE MODULE THAT FIRST NEEDED THEM

They started as private helpers inside the pre-assembler. Once `first_pass` needed the exact same "grab the first whitespace-separated word" logic, duplicating it a second time was still reasonable — but the moment a *third* consumer (`second_pass`) needed it too, that was the signal to stop copy-pasting a genuinely generic string utility and promote it into the shared parsing module instead.

`memory_image.h` / `memory_image.c`

The simulated computer memory itself.

Two byte arrays (code and data images) plus the IC/DC position counters. Store a data byte or a full instruction word, overwrite an already-stored instruction (what `second_pass` needs), and the three functions that pack an instruction's fields into its final 32-bit word.

> **WHY THE CODE IMAGE IS INDEXED BY IC − IC_START, BUT THE DATA IMAGE ISN'T**
>
> Instruction addresses start counting at 100, not 0 — using `ic` directly as an array index would permanently waste the array's first 100 slots. Data addresses start at 0 already (`DC_START` is 0), so `dc` already *is* the correct index with no translation needed. The asymmetry between the two is intentional, not an oversight.

> **WHY THE THREE ENCODE FUNCTIONS BUILD THEIR WORD IN UNSIGNED INT**
>
> The identical shift-into-the-sign-bit problem from `num_utils` , showing up again in a new place: an instruction's opcode field occupies bits 31–26, and real opcodes like `call` (32) and `hlt` (63) have their own top bit set within that 6-bit field — shifting them into position on a signed `int` is undefined behavior for the same underlying reason. This time the fix was applied *before* writing any code, purely because the project had already been bitten by this exact bug twice by this point and knew to watch for it: build the word in `unsigned int` , only converting to a plain `int` once, at the very end, to hand back to the caller.

> **WHY THERE'S NO FUNCTION TO "SHIFT" A DATA BYTE'S ADDRESS**
>
> Unlike a symbol's value (which is looked up by name later, so it has to already be numerically correct), a data byte's address is never stored anywhere as its own separate value — it's simply implied by its position in the array. Whoever needs a data byte's real address (the output writer) just computes `ICF + index` on the spot. Nothing to update after the fact, because nothing was ever cached in the first place.

03 The Pipeline, Stage by Stage

Unlike the foundational modules above, these four run in a real, meaningful sequence — each one depends on the last having finished. `assembler.c`, the entry point, drives all four for every input file.

STAGE 1

`preassembler.h` / `preassembler.c`

Expand every macro call, before real assembly begins.

Reads the raw `.as` file line by line and, per line: if currently inside a macro's body, either close it (`mcroend`) or store the line as body text; otherwise, if the line invokes an already-defined macro, write out its whole stored body in place of this line; otherwise, if the line starts a new `mcro` definition, validate its name and register it; otherwise, write the line through unchanged. Writes the result to a `.am` file — only if the whole file had zero macro-related errors.

> **WHY THE READ BUFFER IS 500 BYTES WHEN A LINE IS CAPPED AT 80**
>
> Detecting an overlong line with a small, exactly-sized buffer is surprisingly fiddly: if a line doesn't fit, you have to keep reading and discarding chunks just to find where the *next* real line begins, without losing sync. An oversized buffer sidesteps that complexity entirely — any real line, even a deliberately too-long test line, always fits in one read. The only assumption made is that no single source line is absurdly long, which is completely safe for this project's actual inputs.

> **WHY "CURRENTLY INSIDE A MACRO" IS CHECKED BEFORE "IS THIS AN INVOCATION"**
>
> If a macro's own body happened to contain a line whose first word matched a *different*, already-defined macro's name, checking for invocations first would wrongly expand that other macro right there, instead of correctly storing the line as literal body text for the macro currently being defined. Checking "are we inside a macro" first closes that off entirely.

WHY FREE_MACRO_TABLE() RUNS AS LITERALLY THE FIRST LINE OF THIS FUNCTION

The assembler processes multiple input files in one run, and macros are scoped to a single file. This call was missing for a while during development — a real bug that would have let a macro defined in one file silently persist into the next file's expansion. Resetting state as the very first action, rather than trusting whoever calls this function to remember to do it, closed that off.

WHY A STRAY "MCRO" MID-LINE GETS ITS OWN DEDICATED ERROR

A line like `hello mcro GEN_MC` (where `mcro` isn't the first word) was originally left uncaught here on purpose — it would still get flagged eventually, just later, when `first_pass` tried to parse `hello` as an unknown operation name. That's technically correct but buries the real mistake behind whatever *other* generic error the line happens to produce. A dedicated check — does `mcro` / `mcroend` appear anywhere in the line other than as its first word? — gives a precise, immediate message instead.

STAGE 2

`first_pass.h` / `first_pass.c`

Discover every label's address, and encode as much of the program as a single read allows.

The largest module, because it's where every foundational module gets used together. Per line: skip empty/comments; pull off an optional label and validate it (syntax, and that it isn't a reserved word); for a directive, store the described data and record the label's address as wherever the data counter was *before* that line added anything; for an instruction, look up the opcode, split the operands, and dispatch to one of seven small encoders depending on the instruction's exact operand shape.

GROUP	EXAMPLES	SHAPE	RESOLVED NOW?
R-arithmetic	add, sub, and, or, nor	3 regs, syntax order = rs, rt, rd	Yes
R-copy	move, mvhi, mvlo	2 regs, *inverted*: rd=2nd operand, rs=1st	Yes
I-immediate	addi…nori, lb…sh	reg, number, reg	Yes
I-branch	beq, bne, blt, bgt	2 regs + a label	**Never** — always deferred
J-jmp	jmp	register OR label form	Register form only
J-label-only	la, call	always a label	Never
J-none	hlt	no operands	Trivially, yes

WHY MOVE/MVHI/MVLO HAVE AN "INVERTED" FIELD MAPPING

Every arithmetic instruction maps its operands straightforwardly, in the order they're written: first operand → rs, second → rt, third → rd. The copy instructions look backwards by comparison — the *first* written operand (the destination) goes into the `rs` field, and the *second* (the source) goes into `rd`, with `rt` left unused. This wasn't guessed; it was traced directly from the assignment's own worked example (`move $23, $2` puts 23 in `rs` and 2 in `rd`) and double-checked against a real test file before being trusted.

WHY BRANCHES ALWAYS DEFER, EVEN TO A LABEL ALREADY DEFINED EARLIER

It would be reasonable to guess that a *backward* reference (the label already exists by the time you reach the branch) could be resolved immediately, while only a forward reference needs deferring. The assignment's own worked example rules this out directly: it includes a branch to a label defined earlier in the file, and even that one is shown still unresolved after the first pass. So the real rule isn't "forward vs. backward" — it's a blanket rule that *every* branch distance and *every* label-form address waits for the second pass, no exceptions, which is simpler to implement correctly than a rule with a special case.

WHY A DATA LABEL'S VALUE IS CAPTURED BEFORE STORING ANY OF THAT LINE'S BYTES

A label on a data line is supposed to point at the address of the *first* byte of that data. If the label's value were recorded after the loop that stores the values, the data counter would have already moved past the start — so `get_dc()` has to be called once, before the storage loop begins, and that captured value is what gets recorded for the label.

WHY LABELS ARE CHECKED AGAINST IS_RESERVED_WORD, NOT JUST SYNTAX

This was an actual gap found during review, not something planned from the start: labels were validated for legal *syntax* (starts with a letter, alphanumeric after, right length) but nothing stopped a label from being named exactly `add` or `asciz`. The assignment explicitly lists this as something that has to be checked, so the same reserved-word check already used for macro names was added here too.

STAGE 3

`second_pass.h` / `second_pass.c`

Resolve every branch distance and label/external address the first pass had to leave blank.

Re-reads the same `.am` file from the top. For a branch, computes the distance from the current instruction to its target and rebuilds the whole instruction word. For a label-form jump/ `la` / `call`, resolves the target's address — unless the target is external, in which case the address field stays 0 and the usage gets recorded in a small list instead (for the `.ext` file). Also handles `.entry`, which the first pass ignored entirely.

> **WHY IT TRACKS ITS OWN CURRENT_IC INSTEAD OF ASKING MEMORY_IMAGE**
>
> By the time `second_pass` runs, `memory_image` 's own instruction counter has already reached its *final* value — it finished moving the moment the first pass ended. It can no longer answer "what address is line 12 at?" So `second_pass` keeps its own local counter, starting at 100 and walking forward the same way the first pass did, re-deriving each line's address itself.

> **WHY SEVERAL CHECKS HERE QUIETLY DO NOTHING INSTEAD OF REPORTING AN ERROR**
>
> A handful of validations in this stage — "is this a real opcode," "does the operand count match" — are exact re-derivations of checks the first pass already performed. Since `second_pass` only ever runs on a file that finished the first pass with zero errors, those specific conditions can never actually occur in practice. The checks are kept anyway, purely so that if that assumption is ever wrong, the code fails safely instead of crashing on a null pointer — but they deliberately don't print anything, so the user is never shown the same mistake twice.

> **WHY A BRANCH REJECTS AN EXTERNAL TARGET BUT A JUMP DOESN'T**
>
> A branch's field is a computed *distance* — relative to where the current instruction sits in this file's own layout, which is meaningless for a label whose real address lives in a file this program never even opens. A jump-style instruction's field is just an *address slot*, which can legitimately stay at 0 and get filled in later during linking (a stage outside this project). That structural difference is exactly why one rejects externals as an error and the other quietly records them for later instead.

STAGE 4

`output_writer.h` / `output_writer.c`

Turn the finished in-memory result into the three files the assignment asks for.

Only runs once both passes have finished with zero errors. Writes `.ob` (a header line, then the code and data images as hex rows), `.ent` (one line per symbol marked as an entry point), and `.ext` (one line per recorded external-symbol usage) — the latter two only if they'd actually have any content.

> **WHY A CODE ROW IS ALWAYS 4 BYTES WIDE, BUT A DATA ROW CAN BE SHORTER**
>
> Every instruction is exactly 4 bytes, so the total code segment length is guaranteed to be an exact multiple of 4 — which also means no output row can ever straddle the boundary between the code and data segments, since that boundary always falls exactly on a 4-byte mark. The data segment has no equivalent guarantee (a string or a run of bytes can add up to any total), so only its very last row might come out short, and the code is written to only need that special case in one place.

> **WHY .ENT AND .EXT USE "WRITE IT, THEN DELETE IT IF EMPTY"**
>
> Rather than first scanning ahead to check whether anything qualifies for the file and only opening it if so, both writers just write speculatively and delete the file afterward if nothing ended up in it. This is the exact same technique the pre-assembler already needed, for its own reason (discarding an incomplete `.am` file on error) — reusing one known-working pattern instead of inventing a second way to solve the same shape of problem.

ENTRY POINT

`assembler.c` (contains main)

Runs the whole pipeline, for every input file.

For each command-line argument: reset the error counter, work out the `.am` / `.ob` / `.ent` / `.ext` filenames from the input name, then run pre-assembler → first pass → second pass → output writer in order, stopping the moment any stage fails.

> **WHY FILENAME CONSTRUCTION LIVES HERE, NOT IN ANY ONE STAGE**
>
> Every pipeline stage just receives an exact filename to read or write — none of them decide what that filename *should be*. Deciding that (strip `.as`, append the right extension) is a single responsibility that belongs to whichever piece of code is already orchestrating all four stages for one file, rather than being duplicated across stages that shouldn't need to know about each other's naming conventions.

> **WHY EVERY STAGE RESETS ITS OWN STATE, INSTEAD OF MAIN DOING IT CENTRALLY**
>
> First_pass owns the job of building the symbol table, so it made more sense for first_pass itself to clear out whatever a previous file left behind, right as it starts, rather than requiring `main` to remember to call a reset function on that stage's behalf before every run. Each module is responsible for its own clean slate.